AssetTrust v0.54
Git-history security hardening

Engineering checkpoint | 11 November 2026 | Original repository audit still required

RELEASE DECISION: NO-GO - NOT PRODUCTION READY

Executive assessment

Version 0.54 replaces a slow per-object Git-history scanner with bounded batch inspection. It reviews historical blobs, commit messages and locally retained reflog commits, including deleted-file credentials that are invisible in the current source tree. The scanner returns finding categories and abbreviated Git object IDs without printing secret values.

Validated new controls

Control	Verified behavior in local tests
Deleted and reset commits	A planted test token remained detectable after deletion and after branch reset via local reflog history.
Historical commit messages	A test token embedded in a commit message was detected without exposing its value.
Incomplete history	Shallow clones, missing .git metadata, oversized content and scan-budget overruns cannot be reported as clean.
Original Git clone support	The new --repo option scans an explicitly chosen existing Git repository, separate from a downloaded source ZIP.
CI enforcement	The parsed GitHub workflow requires the v0.54 version suite and the batched history scan.

Scope note

The actual AssetTrust repository history was not present in this environment. The current source ZIP intentionally returns a blocked history-scan result. No real credential was scanned, revoked or rotated. An independent history scanner and provider-level exposure review are still required.

Test results and release evidence

Automated checks executed against the v0.54 source snapshot. Green static tests do not establish live production safety.

Check	Evidence / result
New Git scanner security suite	15 of 15 passed, using real disposable Git repositories.
Historical and current version suites	52 of 52 passed, v0.03 through v0.54.
CI, architecture and syntax	All passed; workflow parsed successfully with Ruby/Psych.
Migration immutability	25 of 25 SQL migration files identical to v0.53. No new migration.
Snapshot source secret scan	Passed; no high-confidence matches in 2,234 inspected files at recorded checkpoint.
Supervised release plan	89 validation stages planned; NOT executed as a complete live run.
Full Git-history scan on source ZIP	BLOCKED by missing .git, correctly and intentionally.

Unresolved release blockers

Release gate	Status
Genuine npm lockfile; npm ci	MISSING / FAILED; no trusted npm dependency graph available locally.
API / web build and unit runtime tests	FAILED / BLOCKED on missing NestJS, Node types and other packages.
Real ClamAV daemon	NOT AVAILABLE; live benign/EICAR tests have not passed.
PostgreSQL RLS / two-API / HTTP integration	NOT RUN; no disposable live PostGIS and running API deployment.
Authentication browser journeys, TLS, SSH, restore	NOT RUN; requires actual controlled infrastructure.
Original repository secret history	NOT RUN; only disposable test repos scanned.

v0.41 release preflight: 3 of 11 prerequisites present. Production release remains blocked.

Kali validation and handover

Required next actions on the trusted development machine

1. Scan the actual Git clone

From the extracted v0.54 source directory, run the command below against the original repository. Do not place private Git metadata inside a public ZIP.

```
npm run security:scan:v54 -- \
  --repo "$HOME/Projects/assettrust-github-v22/assettrust-v22"
```

If a real token appears in history, revoke and rotate it with the issuing provider first, review access logs and plan any repository cleanup with collaborators. Removing a commit does not revoke leaked credentials.

2. Establish a trusted build

Review root and workspace manifests. Generate and commit a genuine npm lockfile on a connected machine; run npm ci, npm audit, API/web builds and the complete test suite. Do not fake a lockfile or suppress high-severity dependency issues.

3. Validate real security services

Run ClamAV with refreshed signatures and a protected local socket. In a disposable loopback-only PostgreSQL/PostGIS database, run migrations, tenant A/B RLS tests, distributed login checks, two independent API processes, browser flows and authorized HTTP tests. Do not run mutating tests against an existing customer database.

4. Complete independent release review

Verify production HTTPS, HSTS, SSH and firewall exposure; database least privilege; MFA/admin permissions; server-side pricing and form validation; cookies/CSRF; storage and key backups/restoration; full Git history and connected-service secrets; and an independent penetration test.

Limitations

The bundled scanner uses a limited set of high-confidence patterns and locally available Git refs/reflogs. Pruned unreachable objects, remote forks and external secrets are outside its scope. Passing this scanner is not proof that all secrets are safe. See V54_KALI_VALIDATION.md for complete commands and V54_HANDOVER.md for changed files.

FINAL VERDICT: NO-GO - no production or customer-data approval